

Secure Recipes GRETA 92

Projet final formation GRETA 92

Developpement securise d'un site de recettes de cuisine

Otmane Aiboud

PHP · HTML · JavaScript · Tailwind CSS · MySQL

Projet final formation GRETA 92

Developpement securise d'un site de recettes de cuisine

Otmane Aiboud

Stack : PHP, HTML, JavaScript, Tailwind CSS, MySQL

Theme : securite applicative web

Sommaire

1. Contexte
2. Architecture
3. Fonctionnalites
4. Securite applicative
5. Tests de securite realises
6. Conclusion

1. Contexte

Secure Recipes GRETA 92 est un site de recettes de cuisine concu comme une application reelle exposee a Internet. Le public peut consulter les recettes. Les administrateurs authentifies peuvent gerer les recettes et les comptes administrateurs. Le projet applique les protections fondamentales contre XSS, injection SQL, CSRF, brute force et upload de fichiers dangereux.

2. Architecture

- ``public/`` : accueil, detail recette, login, logout, presentation, assets.
- ``admin/`` : dashboard, CRUD recettes, CRUD administrateurs.
- ``public/admin/`` : wrappers compatibles avec une racine web ``public/`` pour exposer les URLs ``/admin/...``.
- ``app/config/`` : configuration applicative et connexion PDO.
- ``app/security/`` : sessions, authentication, CSRF, CSP, brute force, upload.
- ``app/repositories/`` : requetes SQL preparees.
- ``app/validation/`` : validation serveur.
- ``docs/`` : rapport Markdown et PDF.
- ``database.sql`` : schema MySQL, donnees de demonstration et admin initial.

3. Fonctionnalites

- Accueil public avec hero, badges securite et liste des recettes.
- Detail recette avec titre, image, description, ingredients et preparation.
- Connexion admin sans inscription publique.
- Dashboard admin avec compteurs et tentatives echouees recentes.

- CRUD recettes avec upload image.
- CRUD administrateurs avec hachage des mots de passe.
- Page presentation type PowerPoint integree.

4. Securite applicative

A. Authentification et hachage des mots de passe

Technique : `password\_hash()` avec Argon2id si disponible, `password\_verify()` et rehash automatique.

Menace : vol ou lecture directe des mots de passe si la base est compromise.

Solution appliquee : les mots de passe admins sont haches via une fonction centrale. Les nouveaux mots de passe utilisent Argon2id quand PHP le supporte, avec fallback compatible. Le login compare le mot de passe saisi avec le hash stocke et peut re-hacher automatiquement un ancien hash après une connexion valide.

Fichiers concernes : `app/security/auth.php`, `app/repositories/AdminRepository.php`, `admin/admins/create.php`, `admin/admins/edit.php`, `public/login.php`.

Extrait reel :

```
function admin_password_hash(string $password): string
{
    if (defined('PASSWORD_ARGON2ID')) {
        return password_hash($password, PASSWORD_ARGON2ID, [
            'memory_cost' => 65536,
            'time_cost' => 3,
            'threads' => 1,
        ]);
    }

    return password_hash($password, PASSWORD_DEFAULT);
}

$valid = $admin && password_verify($password, (string) $admin['password_hash']);

if (admin_password_needs_rehash((string) $admin['password_hash'])) {
    $repo->updatePasswordHash((int) $admin['id'], admin_password_hash($password));
}
```

Explication : `password\_hash()` applique un algorithme lent adapté aux mots de passe. Argon2id augmente le coût mémoire pour compliquer les attaques hors-ligne. `password\_verify()` évite toute comparaison manuelle. Le rehash permet de moderniser progressivement un hash bcrypt existant sans bloquer l'administrateur.

Limite restante : les identifiants de demonstration doivent etre changes en production.

B. Sessions securisees

Technique : cookies securises et regeneration d'identifiant de session.

Menace : fixation de session et vol de cookie.

Solution appliquee : session `HttpOnly`, `SameSite=Lax`, `Secure` si HTTPS et regeneration apres connexion. En production, une requête HTTP en GET/HEAD est redirigée vers HTTPS ; une requête POST non HTTPS est refusée pour éviter de rejouer un formulaire sensible sur une URL différente.

Fichiers concernés : `app/security/auth.php`, `app/security/headers.php`.

Extrait reel :

```
session_set_cookie_params([
    'lifetime' => 0,
    'path' => '/',
    'domain' => '',
    'secure' => $https,
    'httponly' => true,
    'samesite' => 'Lax',
]);

function login_admin(array $admin): void
{
    session_regenerate_id(true);
    $_SESSION['admin_id'] = (int) $admin['id'];
}

function enforce_https_in_production(): void
{
    if (!is_production() || request_is_https() || PHP_SAPI === 'cli') {
        return;
    }

    $method = strtoupper((string) ($_SERVER['REQUEST_METHOD'] ?? 'GET'));
    if (!in_array($method, ['GET', 'HEAD'], true)) {
        http_response_code(403);
        echo 'HTTPS requis en production.';
        exit;
    }

    $host = preg_replace('/^[A-Za-z0-9.-:]/', '', (string) ($_SERVER['HTTP_HOST'] ?? ''));
    $requestUri = str_replace(["\r", "\n"], '', (string) ($_SERVER['REQUEST_URI'] ?? ''));
    header('Location: https://'. $host . $requestUri, true, 301);
    exit;
}
```

Limite restante : en production réelle, il faut aussi configurer HTTPS côté hébergeur avec un certificat valide.

C. Protection XSS

Technique : echappement HTML centralise.

Menace : execution de JavaScript injecte dans une recette ou un compte admin.

Solution appliquee : toutes les donnees affichees depuis la base passent par `e()`.

Fichiers concernés : `app/helpers/functions.php`, `public/index.php`, `public/recipe.php`, `admin/\*`.

Extrait reel :

```
function e(mixed $value): string
{
    return htmlspecialchars((string) $value, ENT_QUOTES, 'UTF-8');
}

<h1 class="mt-5 text-4xl font-bold text-white"><?= e($recipe['title']) ?></h1>
```

Limite restante : si un jour du HTML enrichi est autorise, il faudra ajouter une liste blanche stricte.

D. Content Security Policy

Technique : header CSP centralisé avec ****nonce par requête**** pour autoriser uniquement les scripts inline explicitement étiquetés.

Menace : exécution de scripts externes, injection de contenu actif, exfiltration via inline script forgé par un attaquant.

Solution appliquée : CSP ``default-src 'self`, `script-src 'self' 'nonce-{nonce}``, styles autorisés sur ``self`` + Google Fonts, fonts autorisées sur ``self` data:`` + Google Fonts, images locales/data, objets interdits, framing interdit, ``form-action 'self``.

Le nonce est généré via ``random_bytes(16)`` à chaque requête (helper ``csp_nonce()``) et n'est jamais réutilisé. Les rares scripts inline indispensables (JSON-LD ``Recipe``) portent l'attribut ``nonce="<?= e(csp_nonce()) ?>"`` ; tout script inline non noncé est rejeté par le navigateur.

Fichiers concernés : ``app/security/headers.php``, ``app/bootstrap.php``, ``public/recipe.php`` (JSON-LD avec nonce).

Extrait réel :

```
function csp_nonce(): string
{
    static $nonce = null;
    if ($nonce === null) {
        $nonce = base64_encode(random_bytes(16));
    }
    return $nonce;
}

function apply_security_headers(): void
{
    if (headers_sent()) return;
    $nonce = csp_nonce();
    header(
        "Content-Security-Policy: default-src 'self'; "
        . "script-src 'self' 'nonce-{$nonce}'; "
        . "style-src 'self' https://fonts.googleapis.com; "
        . "img-src 'self' data; "
        . "font-src 'self' data: https://fonts.gstatic.com; "
        . "object-src 'none'; base-uri 'self'; "
        . "frame-ancestors 'none'; form-action 'self'"
    );
}

// public/recipe.php - JSON-LD Recipe SEO avec nonce CSP
echo '<script type="application/ld+json" nonce="' . e(csp_nonce()) . '">'
    . json_encode($jsonLd, JSON_UNESCAPED_UNICODE | JSON_UNESCAPED_SLASHES)
    . '</script>';
```

Limite restante : vérifier les headers depuis l'URL finale Railway après déploiement.

E. Protection injection SQL

Technique : PDO prepare/execute.

Menace : modification d'une requete SQL par saisie utilisateur.

Solution appliquee : les repositories utilisent des requetes preparees et des parametres.

Fichiers concernes : ``app/repositories/RecipeRepository.php``, ``app/repositories/AdminRepository.php``, ``app/repositories/LoginAttemptRepository.php``.

Lecture :

```
$stmt = $this->pdo->prepare('SELECT * FROM recipes WHERE slug = :slug LIMIT 1');  
$stmt->execute(['slug' => $slug]);
```

Creation :

```
$stmt = $this->pdo->prepare(  
    'INSERT INTO recipes (title, slug, short_description, description, ingredients,  
    preparation_steps, image_path, created_at, updated_at)  
    VALUES (:title, :slug, :short_description, :description, :ingredients,  
    :preparation_steps, :image_path, NOW(), NOW())'  
);
```

Modification :

```
$stmt = $this->pdo->prepare(  
    'UPDATE admins SET username = :username, email = :email, updated_at = NOW() WHERE id  
    = :id'  
);
```

Suppression :

```
$stmt = $this->pdo->prepare('DELETE FROM recipes WHERE id = :id');  
$stmt->execute(['id' => $id]);
```

Limite restante : utiliser en production un utilisateur MySQL dedie avec droits limites, comme indique dans `database.sql`.

F. Protection CSRF

Technique : token aleatoire en session, champ cache et verification serveur.

Menace : forcer un admin connecte a executer une action sensible.

Solution appliquee : les formulaires de connexion, creation, modification et suppression incluent un token.

Fichiers concernes : `app/security/csrf.php`, `public/login.php`, `admin/recipes/\*`, `admin/admins/\*`.

Extrait reel :

```
function generate_csrf_token(): string  
{  
    if (empty($_SESSION['csrf_token'])) {  
        $_SESSION['csrf_token'] = bin2hex(random_bytes(32));  
    }  
  
    return (string) $_SESSION['csrf_token'];  
}  
  
function verify_csrf_token(?string $token): bool  
{  
    return is_string($token)  
        && isset($_SESSION['csrf_token'])  
        && hash_equals((string) $_SESSION['csrf_token'], $token);  
}
```

Limite restante : une rotation de token par formulaire pourrait etre ajoutee pour des exigences plus strictes.

G. Protection brute force

Technique : table `login\_attempts`, comptage des echecs recents.

Menace : tentative automatisee de deviner le mot de passe admin.

Solution appliquee : apres 5 echecs sur 15 minutes par email ou IP, la connexion est temporairement refusee.

Fichiers concernes : `app/security/brute\_force.php`, `app/repositories/LoginAttemptRepository.php`, `public/login.php`.

Extrait reel :

```
const MAX_LOGIN_FAILURES = 5;
const LOGIN_WINDOW_MINUTES = 15;
const LOGIN_BLOCK_MINUTES = 15;

$failures = $repo->countRecentFailures($email, client_ip(), LOGIN_WINDOW_MINUTES);
return $failures >= MAX_LOGIN_FAILURES;
```

Limite restante : un nettoyage automatique des anciennes tentatives peut etre planifie.

H. Upload securise des images

Technique : extension, MIME, taille, nom aleatoire et blocage execution.

Menace : televersement de fichier executable ou fichier deguise.

Solution appliquee : seuls `jpg`, `jpeg`, `png`, `webp` sont acceptes. Le MIME est verifie avec `finfo`. Le nom original n'est jamais reutilise.

Fichiers concernes : `app/security/upload.php`, `public/uploads/recipes/.htaccess`.

Extrait reel :

```
$allowedExtensions = ['jpg', 'jpeg', 'png', 'webp'];
if (!in_array($extension, $allowedExtensions, true)) {
    return ['path' => null, 'error' => 'Extension refusee. Formats acceptes : jpg, jpeg, png, webp.'];
}

$finfo = new finfo(FILEINFO_MIME_TYPE);
$mime = $finfo->file($tmpPath) ?: '';

$filename = bin2hex(random_bytes(16)) . '.' . $extension;
```

Limite restante : stocker hors webroot et servir via script controle serait encore plus robuste.

I. Protection des acces admin

Technique : middleware `require\_admin()`.

Menace : acces direct aux pages `/admin` sans authentication.

Solution appliquee : chaque page admin appelle `require\_admin()` avant d'afficher le contenu.

Fichiers concernés : `app/security/auth.php`, `admin/\*`.

Extrait reel :

```
function require_admin(): void
{
    if (!is_admin_authenticated()) {
        flash('error', 'Acces reserve aux administrateurs authentifies.');
```

```
        redirect('/login.php');
    }
}
```

Limite restante : ajouter un controle serveur web pour refuser l'indexation des URLs admin.

J. Validation des entrees

Technique : validation serveur par type de formulaire.

Menace : donnees incompletes, trop longues ou non conformes.

Solution appliquee : les recettes et admins sont controles avant toute insertion ou modification.

Fichiers concernés : `app/validation/recipe\_validation.php`, `app/validation/admin\_validation.php`.

Extrait reel :

```
if ($title === '' || mb_strlen($title) > 150) {
    $errors['title'] = 'Le titre est obligatoire et limite a 150 caracteres.';
}

if (!filter_var($email, FILTER_VALIDATE_EMAIL) || mb_strlen($email) > 190) {
    $errors['email'] = 'Email administrateur invalide.';
}
```

Limite restante : ajouter des tests automatisés de validation.

K. Protection contre l'auto-suppression d'administrateur

Technique : double protection UI + serveur.

Menace : un administrateur (ou un attaquant ayant volé sa session) supprime son propre compte, provoquant un déni de service applicatif sur le back-office.

Solution appliquée :

- **Côté UI** : le bouton « Supprimer » est remplacé par un badge inerte « ● Vous » sur la ligne correspondant au compte connecté. L'admin ne peut donc pas déclencher l'action depuis l'interface.
- **Côté serveur** : `admin/admins/delete.php` rejette toute requête où `id` correspond à `\$\_SESSION['admin\_id']`, indépendamment de l'origine (formulaire forgé, curl, etc.).
- **Garde-fou supplémentaire** : la suppression du dernier administrateur restant est également bloquée par ` \$repo->count() <= 1 `.

Fichiers concernés : `app/security/auth.php` (helper `current\_admin\_id()`), `admin/admins/index.php` (UI), `admin/admins/delete.php` (serveur).

Extrait réel :

```
// app/security/auth.php
function current_admin_id(): int
{
    return (int) ($_SESSION['admin_id'] ?? 0);
}

// admin/admins/delete.php
if ($repo->count() <= 1) {
    flash('error', 'Impossible de supprimer le dernier administrateur.');
```

redirect('/admin/admins/index.php');

```
}
if ($id === (int) ($_SESSION['admin_id'] ?? 0)) {
    flash('error', 'Suppression de votre propre compte refusee pendant la session active.');
```

redirect('/admin/admins/index.php');

```
}

// admin/admins/index.php - UI conditionnelle
if ((int) $admin['id'] === $currentAdminId): ?>
    <span class="badge-self" title="Vous ne pouvez pas supprimer votre propre compte.">●
Vous</span>
<?php else: ?>
    <form method="post" action="/admin/admins/delete.php" data-confirm="Êtes-vous sûr de
vouloir supprimer définitivement l'administrateur « <?= e($admin['username']) ?> » ?">
        <?= csrf_field() ?>
        <input type="hidden" name="id" value="<?= e($admin['id']) ?>">
        <button class="btn-danger" type="submit">Supprimer</button>
    </form>
<?php endif;
```

Limite restante : un audit log persistant (qui a tenté de supprimer qui, quand, depuis quelle IP) renforcerait la traçabilité.

L. Confirmations explicites pour les actions destructives

Technique : pattern `data-confirm` + modale custom avec focus trap.

Menace : suppression accidentelle de recette ou d'administrateur (clic mal placé, raccourci clavier).

Solution appliquée : tout `<form>` exécutant une action destructive porte un attribut `data-confirm` contenant un message qui inclut **l'identité précise de l'élément** (titre de recette, nom + email d'admin). Un handler JS injecté par `admin\_footer()` (`assets/js/admin.js`) intercepte le `submit`, ouvre une modale `role="alertdialog"` avec focus trap, gestion `Escape` et `Enter`, et résout le submit uniquement après confirmation explicite.

La modale est construite par DOM API (`document.createElement` + `textContent`) — aucun `innerHTML` avec contenu dynamique, donc zéro surface d'XSS même si le message contient des caractères spéciaux.

Fichiers concernés : `public/assets/js/admin.js`, `admin/recipes/index.php`, `admin/admins/index.php`, `app/helpers/functions.php` (`admin\_footer` charge `admin.js`).

Extrait réel :

```
// admin/recipes/index.php
<form method="post" action="/admin/recipes/delete.php"
    data-confirm="Êtes-vous sûr de vouloir supprimer définitivement la recette « <?=
e($recipe['title']) ?> » ? Cette action est irréversible.">
    <?= csrf_field() ?>
    <input type="hidden" name="id" value="<?= e($recipe['id']) ?>">
    <button class="btn-danger" type="submit">Supprimer</button>
</form>
```

```
```js
```

```
// public/assets/js/admin.js — extrait
```

```
document.querySelectorAll('form[data-confirm]').forEach(function (form) {
```

```
var confirmed = false;
```

```
form.addEventListener('submit', function (event) {
```

```
if (confirmed) return;
```

```
event.preventDefault();
```

```
openConfirm(form.getAttribute('data-confirm')).then(function (ok) {
```

```
if (ok) { confirmed = true; form.submit(); }
```

```
});
```

```
});
```

```
});
```

Limite restante : étendre le pattern aux mises à jour à risque (changement d'email admin, par exemple) si nécessaire.

### M. Notes et commentaires publics modérés

Technique : tables séparées `recipe\_ratings` et `recipe\_comments`, CSRF sur les formulaires publics, validation serveur, échappement à l'affichage et modération admin.

Menace : XSS via commentaires, spam public, modification non autorisée des avis, pollution du front-office.

Solution appliquée : les notes utilisent une empreinte visiteur hashée (`public\_actor\_hash`) avec une clé unique par recette/visiteur. Les commentaires publics sont insérés avec le statut `pending` et ne sont affichés côté front-office que lorsqu'un administrateur les passe en `approved`. Les actions admin de modération passent par POST et CSRF.

Fichiers concernés : `public/recipe.php`,  
`app/repositories/RecipeInteractionRepository.php`, `admin/comments/index.php`,  
`database.sql`.

Extrait réel :

```
php
```

```
// public/recipe.php — commentaire public en attente
```

```
if ($authorName === "" || mb_strlen($authorName) > 80) {
```

```
flash('error', 'Le nom est obligatoire et limite a 80 caracteres.');
```

```
} elseif (mb_strlen($content) < 5 || mb_strlen($content) > 800) {
```

```
flash('error', 'Le commentaire doit contenir entre 5 et 800 caracteres.');
```

```
} else {
```

```
$interactionRepo->createComment(((int) $recipe['id'], $authorName, $content, public_actor_hash());
```

```
flash('success', 'Commentaire envoye. Il apparaitra apres validation.');
```

```
}
```

```
php
```

```
// app/repositories/RecipeInteractionRepository.php — affichage public modéré
```

```
public function approvedComments(int $recipeId): array
```

```
{
```

```
$stmt = $this->pdo->prepare(
```

```
"SELECT * FROM recipe_comments
```

```
WHERE recipe_id = :recipe_id AND status = 'approved'
```

```
ORDER BY created_at DESC, id DESC"
```

```
);
```

```
$stmt->execute(['recipe_id' => $recipeId]);
```

```
return $stmt->fetchAll();
```

```
}
```

Limite restante : ajouter une limitation de fréquence spécifique aux commentaires si le site reçoit beaucoup de trafic public.

### N. Timeout de session et journal de sécurité

Technique : expiration d'inactivité côté session PHP et table `security\_logs`.

Menace : session administrateur laissée ouverte sur un poste partagé, absence de traçabilité sur les actions sensibles.

Solution appliquée : `require\_admin()` vérifie l'âge de la dernière activité et coupe la session admin après 30 minutes d'inactivité. Les événements importants (connexion, échec, commentaire public, modération, duplication/suppression recette) sont journalisés en base avec type, email, IP, user-agent et détail. Une page admin dédiée permet ensuite de filtrer ce journal par type, recherche libre et plage de dates, de le paginer, de l'exporter en CSV et de nettoyer les anciennes entrées. Un script CLI permet aussi d'automatiser le nettoyage via une tâche planifiée.

Fichiers concernés : `app/security/auth.php`,  
`app/repositories/SecurityLogRepository.php`,  
`app/repositories/LoginAttemptRepository.php`, `app/helpers/functions.php`,  
`admin/dashboard.php`, `admin/security-logs/index.php`,  
`scripts/cleanup\_security\_data.php`, `database.sql`.

Extrait réel :

php

```
// app/security/auth.php — timeout admin
```

```
const ADMIN_SESSION_TIMEOUT_SECONDS = 1800;
```

```
function enforce_admin_session_timeout(): void
```

```
{
```

```
if (!isset($_SESSION['admin_id'])) {
```

```
return;
```

```
}
```

```
$lastActivity = (int) ($_SESSION['admin_last_activity'] ?? time());
```

```
if (time() - $lastActivity <= ADMIN_SESSION_TIMEOUT_SECONDS) {
```

```
return;
```

```
}
```

```
unset($_SESSION['admin_id'], $_SESSION['admin_email'], $_SESSION['admin_username'],
$_SESSION['admin_last_activity']);
```

```
session_regenerate_id(true);
```

```
flash('error', 'Session expirée après inactivité. Merci de vous reconnecter.');
```

```
}
```

php

```
// app/helpers/functions.php — journal non bloquant
```

```

function record_security_event(PDO $pdo, string $eventType, string $details, ?string $actorEmail = null): void
{
 try {
 (new SecurityLogRepository($pdo))->create([
 'event_type' => substr($eventType, 0, 80),
 'actor_email' => $actorEmail ? substr($actorEmail, 0, 190) : null,
 'ip_address' => request_ip(),
 'user_agent' => request_user_agent(),
 'details' => substr($details, 0, 1000),
]);
 } catch (Throwable) {
 // Le journal ne doit jamais bloquer une action utilisateur.
 }
}

php
// app/repositories/SecurityLogRepository.php — filtrage prepare
public function filtered(array $filters = [], int $limit = 20, int $offset = 0): array
{
 [$where, $params] = $this->filteredQueryParts($filters);

 $sql = 'SELECT * FROM security_logs' . $where . ' ORDER BY created_at DESC, id DESC LIMIT :limit OFFSET :offset';

 $stmt = $this->pdo->prepare($sql);

 foreach ($params as $name => $value) {
 $stmt->bindValue($name, $value);
 }

 $stmt->bindValue(':limit', $limit, PDO::PARAM_INT);
 $stmt->bindValue(':offset', $offset, PDO::PARAM_INT);

 $stmt->execute();

 return $stmt->fetchAll();
}

private function filteredQueryParts(array $filters): array
{
 $dateFrom = $this->normalizedDate((string) ($filters['date_from'] ?? ''));

 if ($dateFrom !== '') {
 $conditions[] = 'created_at >= :date_from';
 $params[':date_from'] = $dateFrom . ' 00:00:00';
 }

 return [$conditions ? ' WHERE ' . implode(' AND ', $conditions) : '', $params];
}

```

```

}

php
// admin/security-logs/index.php — export CSV réservé aux admins
if (!$error && (string) ($_GET['export'] ?? '') === 'csv') {
 $exportLogs = $securityLogRepo->filtered($filters, 5000, 0);
 header('Content-Type: text/csv; charset=UTF-8');
 header('Content-Disposition: attachment; filename="' . $filename . '"');
 $output = fopen('php://output', 'w');
 fputcsv($output, ['id', 'event_type', 'actor_email', 'ip_address', 'user_agent', 'details', 'created_at'], ',', '"', "\n");
}

```

```

php
// scripts/cleanup_security_data.php — maintenance planifiable
$oldAttempts = $loginAttemptRepo->countOlderThanDays($days);
$oldLogs = $securityLogRepo->countOlderThanDays($days);

if (!$dryRun) {
 $deletedAttempts = $loginAttemptRepo->deleteOlderThanDays($days);
 $deletedLogs = $securityLogRepo->deleteOlderThanDays($days);
 $securityLogRepo->create(['event_type' => 'maintenance_cleanup']);
}
...

```

Limite restante : envoyer ces logs vers un service externe en production pour éviter qu'un attaquant ayant accès à la base puisse les effacer.

## 5. Tests de securite realises

- Tentative XSS dans titre recette : affichée comme texte avec `e()``.
- Tentative SQLi dans login : requête préparée, pas de concaténation SQL.
- Suppression recette sans CSRF : refusée par `require\_valid\_csrf()``.
- Accès `/admin/dashboard.php` sans connexion : redirection login.
- Upload fichier `.php` : extension refusée.
- Upload image trop lourde : limite 2 Mo.
- Plusieurs échecs login : blocage après 5 échecs récents.
- Mot de passe stocké : hash présent dans `database.sql`, aucun mot de passe clair en table.
- **CSP avec nonce vérifiée** : `curl -I` montre un nonce différent à chaque requête.
- **Script inline sans nonce refusé** par le navigateur (vérifié dans la console DevTools).
- **Tentative d'auto-suppression admin** (curl POST avec id du compte courant) : refusée serveur avec flash explicite.
- **Confirmation modale** : Escape annule, Enter confirme, Tab reste dans la modale (focus trap), bouton Annuler n'envoie aucune requête.

- **Commentaire public** : insertion en `pending`, invisible côté public avant approbation admin.
- **Journal sécurité** : duplication recette et login admin créent une entrée `security\_logs`; la page `/admin/security-logs/index.php` filtre par type, recherche, dates et pagine les événements.
- **Export CSV journal** : `/admin/security-logs/index.php?export=csv` renvoie un fichier CSV après authentification admin avec les mêmes filtres que l'écran.
- **Nettoyage journal** : l'action de nettoyage est en POST + CSRF et supprime les logs/tentatives anciennes via requêtes préparées.
- **Maintenance CLI** : `php scripts/cleanup\_security\_data.php --dry-run` compte les entrées anciennes et `php scripts/cleanup\_security\_data.php --days=90` les supprime puis journalise `maintenance\_cleanup`.
- **Tests automatisés PHPUnit** : `composer test` exécute 12 tests / 25 assertions sur hash admin, CSRF, HTTPS, validation serveur, repositories de logs et nettoyage.
- **Timeout session** : la session admin expire après 30 minutes d'inactivité.
- **HTTPS production** : `APP\_ENV=production` redirige les requêtes GET/HEAD HTTP vers HTTPS et refuse les POST HTTP.
- **Argon2id / rehash** : connexion admin valide avec ancien hash déclenche un rehash vers l'algorithme courant si nécessaire.
- **Navigation responsive** : Tailwind compilé localement.

## 6. Conclusion

Secure Recipes GRETA 92 montre une application PHP/MySQL complete, structuree et securisee. Le projet relie chaque fonctionnalite a une protection concrete et documentee. Il reste volontairement simple cote architecture pour que le jury puisse identifier les mecanismes de securite sans abstraction inutile.